

实验9. AdaBoost

实验目的

1. 实现弱分类器（单层决策树，或决策树桩）
2. 实现AdaBoost算法
3. 熟悉scikit-learn中的数据集使用方法
4. 熟悉scikit-learn中AdaBoost分类器的调用

实验数据

scikit-learn中的乳腺癌数据集，选自美国威斯康辛大学医院的乳腺肿瘤就诊者的检测数据（Breast Cancer Wisconsin dataset）。

该数据集包含569条数据，每条数据包含30个特征，包括乳腺肿块的细胞核的各种特征，例如半径、纹理、周长、面积、光滑度、紧凑性、凹度、凹点、对称、分形维数等，这些数据已经过预处理，数值范围在一定区间内，方便进行机器学习算法的处理。

数据标注有两类，一类是良性（标记为0），另一类是恶性（标注为1）。

实验步骤

1. 数据准备

1.1 数据装载

scikit-learn模块中提供了乳腺癌二分类数据集的装载、显示相关信息、数据划分等函数。其读取方式如下：

```
1  from sklearn.datasets import load_breast_cancer
2  from sklearn.model_selection import train_test_split
3
4  breastcancerdata = load_breast_cancer()
5  X = breastcancerdata.data # (569,30)
6  out = breastcancerdata.target # (569,)
7
8  dsDescr = breastcancerdata['DESCR']
9  print(breastcancerdata.DESCR)
10
11 featureNames = breastcancerdata.feature_names
```

```
12 print(featureNames)
```

1.2 数据预处理

面向二分类任务的AdaBoost，数据的输出标签应为+1和-1，因此，需要通过计算将0和1变为±1。请修改如下代码：

代码块

```
1 # 可以采用 Label *2 -1 的方式
2 y = out * 2 - 1
```

1.3 划分训练集和测试集

通过`train_test_split()`将569条标注数据划分为训练集和测试集，各自比例为70%：30%。具体代码如下：

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

此外，还可以通过`make_classification()`方法自动生成你需要的分类数据集。使用示例代码如下（[sklearn.datasets.make_classification — scikit-learn 1.4.2 documentation](#)）：

```
X, y = make_classification(n_samples=1000, n_features=20, n_informative=2, n_redundant=0,
    random_state=7, n_classes=2) # 生成1000个样本，每个样本有20个特征，类别数量为2，其他参数
    信息见官网文档
```

2. 构造弱分类器

AdaBoost（Adaptive Boosting）是一个集成学习算法，它通过结合多个简单的分类器（通常称为弱分类器）来形成一个强大的分类器。在AdaBoost的大多数实现中，弱分类器选的是一层决策树（也被称为决策树桩）。

2.1 决策树桩分类器

决策树桩（或单层决策树）是AdaBoost中最经常使用的弱分类器，其判别模型为 “ $attr > or < threshold$ ”。该模型只有一个属性（即特征）参与决策。通过选择同阈值的比较将样本分为两类。因此，此类分类器由3部分组成，一是属性（或特征）用 `dimension` 表示；二是阈值，使用 `thresh_val`，；三是大于或小于，使用 `thresh_ineq` 表示。

```
1 def stump_classify(X, dimension, thresh_val, thresh_ineq):
2     m, n = X.shape
```

```

3
4     ret_array = np.ones((m, 1))
5     if thresh_ineq == 'lt':
6         ret_array[X[:, dimension] <= thresh_val] = -1.0
7     else:
8         ret_array[X[:, dimension] > thresh_val] = -1.0
9     return ret_array

```

可以使用如下代码对决策树桩分类器进行测试

```

1  X_example = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
2  ''' 通过修改下述三个变量来设置分类器 '''
3  dimension = 0
4  thresh_val = 2
5  thresh_ineq = 'lt'
6
7  result = stump_classify(X_example, dimension, thresh_val, thresh_ineq)
8  print(result)

```

2.2 决策树桩分类器的训练

决策树桩（stump）分类器的训练通过枚举特征 `dimension`、阈值 `thresh_val`、不等式 `thresh_ineq` 的所有组合，对各种组合对应的分类器的预测性能进行比较，进而选出最佳决策树桩，即在训练数据集上预测准确率最高的决策树桩。

具体代码如下：

```

1  """训练决策树桩分类器——选择最佳单层决策树"""
2  def build_stump(X, y, w_data):
3      m, n = X.shape
4      num_steps = 10.0
5      best_stump = {}
6      best_class_est = np.zeros((m, 1))
7      min_error = np.inf # 正无穷大
8
9      for i in range(n): # 第一层循环：枚举所有维，看看哪一维适合做决策树桩
10         range_min = X[:, i].min()
11         range_max = X[:, i].max()
12         step_size = (range_max - range_min) / num_steps
13         for j in range(-1, int(num_steps) + 1): # 第二层循环：枚举当前维的所有候选
            阈值，看看哪个阈值适合做当前决策树桩的阈值
14             for inequal in ['lt', 'gt']: # 第三层循环：枚举“小于”和“大于”两种情况
15                 thresh_val = (range_min + float(j) * step_size)

```

```

16         predicted_vals = '''CODE HERE: 构造决策树桩，并对训练数据集进行预
    测'''
17         err_arr = np.ones((m, 1))
18         err_arr[predicted_vals == y] = 0
19         weighted_error = np.dot(w_data.T, err_arr)
20
21         if weighted_error < min_error: # 选择最优决策树桩
22             min_error = weighted_error
23             best_class_est = predicted_vals.copy()
24             best_stump['dim'] = i
25             best_stump['thresh'] = thresh_val
26             best_stump['ineq'] = inequal
27     return best_stump, min_error, best_class_est

```

返回最佳决策树参数字典 `best_stump`、错误率以及预测结果。

3. 实现AdaBoost

在这部分需要完成 `AdaBoostClassifier_` 类。

3.1 AdaBoost算法的伪代码

AdaBoost算法的核心原理和步骤如下：

Algorithm 7: AdaBoost.M1

Input: 训练数据集 D ，子分类器学习算法，迭代次数 T

Output: $F(x) = \sum_{t=1}^T \alpha_t f_t(x)$

1 初始化各训练样本权重 $w_1^{(i)} = \frac{1}{m}$

2 **for** $t = 1, \dots, T$ **do**

3 使用带权重 $w_t^{(i)}$ 的训练数据集 D 训练得到子分类器 $f_t(x)$

4 计算 $f_t(x)$ 的权重: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$ ，其中，错误率 $\epsilon_t = \frac{\sum_{i=1}^m w_i \mathbb{I}(y^{(i)} \neq f_t(x^{(i)}))}{\sum_{i=1}^m w_i}$ 。当

$\epsilon > 0.5$ 时 $\alpha_t = 0$

5 更新样本权重: $w_{t+1}^{(i)} = \frac{w_t^{(i)}}{Z_t} \exp(-y^{(i)} \alpha_t f_t(x^{(i)}))$ ，其中，
 $Z_t = \sum_{i=1}^m w_t^{(i)} \exp(-y^{(i)} \alpha_t f_t(x^{(i)}))$

6 **end**

3.2 实现AdaBoost分类模型

这是本次实验的主体，设计自己的 `AdaBoostClassifier_` 类。具体地，通过继承scikit-learn中的 `BaseEstimator` 和 `ClassifierMixin` 可以直接集中精力在模型的训练和预测上，也可以让自定义分类器能够与scikit-learn的其他工具和流程无缝集成。

3.2.1 实现fit()方法

在 `__init__()` 中设定了构造弱分类器的数量 `num_iters`，并使用 `self.classifiers` 属性存放每一轮的弱分类器最佳决策树。

我们还参照scikit-learn的接口形式在类内部定义并实现 `fit()`：

```
1  from sklearn.base import BaseEstimator, ClassifierMixin # mixin 混合
2
3  class AdaBoostClassifier_(BaseEstimator, ClassifierMixin):
4      def __init__(self, num_iters=40):
5          self.num_iters = num_iters
6          self.classifiers = []
7
8      def fit(self, X, y):
9          y = y.reshape(-1, 1) # 确保 y 是列向量
10         m = X.shape[0]
11         w = np.ones((m, 1)) / m
12         agg_class_est = np.zeros((m, 1))
13         for i in range(self.num_iters):
14             '''CODE HERE:实现算法第3行，训练出当前的决策树桩'''
15
16             '''CODE HERE:实现算法第4行，先计算错误率，然后计算alpha'''
17
18             best_stump['alpha'] = alpha
19             self.classifiers.append(best_stump)
20             '''CODE HERE:实现算法第5行，更新样本权重w，供下一轮循环使用'''
21
22
23             if error_rate == 0.0:
24                 break
25         return self
```

在第一个需要实现的部分中，需要根据当前的训练样本和个样本的权重来训练最佳决策树桩，其中，训练可以调用上部分实现的 `build_stump()` 函数。

在第二个需要实现的部分中，需要利用 `build_stump()` 返回的错误率 `min_error` 计算该决策树对应的alpha值。alpha的计算公式请见算法第4行。

第三部分需要根据 w_t 、alpha和决策树的预测结果更新样本权重 w_{t+1} 。其中的 `y` 是样本标签，决策树桩的预测结果为 `build_stump()` 返回的 `best_class_est`。第5行算法中的分母 Z_t 中是规范化因子，目的是为了使 w_{t+1} 所有元素和为1。

3.2.2 实现predict()方法

主要是将fit()方法获得 `self.classifiers` 的各决策树桩，使用它们对输入进行预测，并将结果加权求和起来，来给出最后输出。

```
1 class AdaBoostClassifier_(BaseEstimator, ClassifierMixin):
2     def predict(self, X):
3         m = X.shape[0]
4         agg_class_est = np.zeros((m, 1))
5         for classifier in self.classifiers:
6             dim = classifier['dim']
7             thresh = classifier['thresh']
8             ineq = classifier['ineq']
9             alpha=classifier['alpha']
10
11             '''CODE HERE:'''
12
13         return
```

3.2.3 实例化AdaBoost分类器，并使用它

完成了fit()函数，你就可以通过AdaBoostClassifier_类实例化一个分类器对象，获取数据集后即可进行训练和预测。实例化示例代码如下：

```
1 from sklearn.metrics import accuracy_score
2
3 clf = AdaBoostClassifier_(num_iters=10)
4 '''CODE HERE'''
5 print(accuracy_score(y_test, y_pred))
6
7 # 另一种直接度量训练数据上准确率的方法
8 print(clf.score(X_test, y_test))
```

4. 调用scikit-learn中的AdaBoost

scikit-learn模块也提供了AdaBoost类，调用方法与支持向量机实验中使用的方法相同。请自行实例化一个分类器对象，并调用fit()、predict()和score()等方法对威斯康辛乳腺癌数据进行分类。

```
1 from sklearn.ensemble import AdaBoostClassifier
2
3 '''CODE HERE:实例化AdaBoostClassifier的实例，并训练、预测、性能评估'''
4
```

